PROGRAM 4: Implement Document Storage and Top-k Retrieval Using a Vector Database

AIM

To store document embeddings in a FAISS vector database and retrieve the top-k most relevant documents for a given query.

ALGORITHM

1. Install the required libraries.
2. Load a pre-trained embedding model.
3. Prepare documents for storage.
4. Generate embeddings for the documents.
5. Normalize the embeddings and create a FAISS index.
6. Add the embeddings to the index.
7. Accept a query and a value for k.
8. Generate the query embedding.
9. Retrieve the top-k documents from the index.
10. Display the retrieved documents and their scores.

CODE

Cell 1: Install libraries

```
!pip -q install sentence-transformers faiss-cpu
```

Cell 2: Python program

```
import faiss

import numpy as np

from sentence_transformers import SentenceTransformer

# Load model

model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
# Documents
documents = [
    "AI is transforming the healthcare industry.",
    "Python is used to develop web applications.",
    "Machine learning analyzes data to find patterns.",
    "Cybersecurity prevents unauthorized access.",
    "Natural language processing helps computers understand text.",
    "Computer vision enables machines to analyze images."
]
# Generate document vectors
vectors = model.encode(
    documents,
    convert_to_numpy=True
).astype("float32")
# Normalize vectors
faiss.normalize_L2(vectors)
# Create and populate index
dimension = vectors.shape[1]
database = faiss.IndexFlatIP(dimension)
database.add(vectors)
# Query
query = "How can computers understand human language?"
k = 3
query_vector = model.encode(
```

```
    [query],
    convert_to_numpy=True
).astype("float32")
faiss.normalize_L2(query_vector)
# Retrieve top-k documents
scores, indices = database.search(query_vector, k)
print("Query:", query)
print("Top-k:", k)
for rank, (score, idx) in enumerate(
    zip(scores[0], indices[0]), start=1
):
    print(f"\nRank {rank}")
    print("Document:", documents[idx])
    print("Similarity:", round(float(score), 4))
```

INPUT

Query: How can computers understand human language?

k = 3

OUTPUT

Query: How can computers understand human language?

Top-k: 3

Rank 1

Document: Natural language processing helps computers understand text.

Similarity: [calculated score]

Rank 2

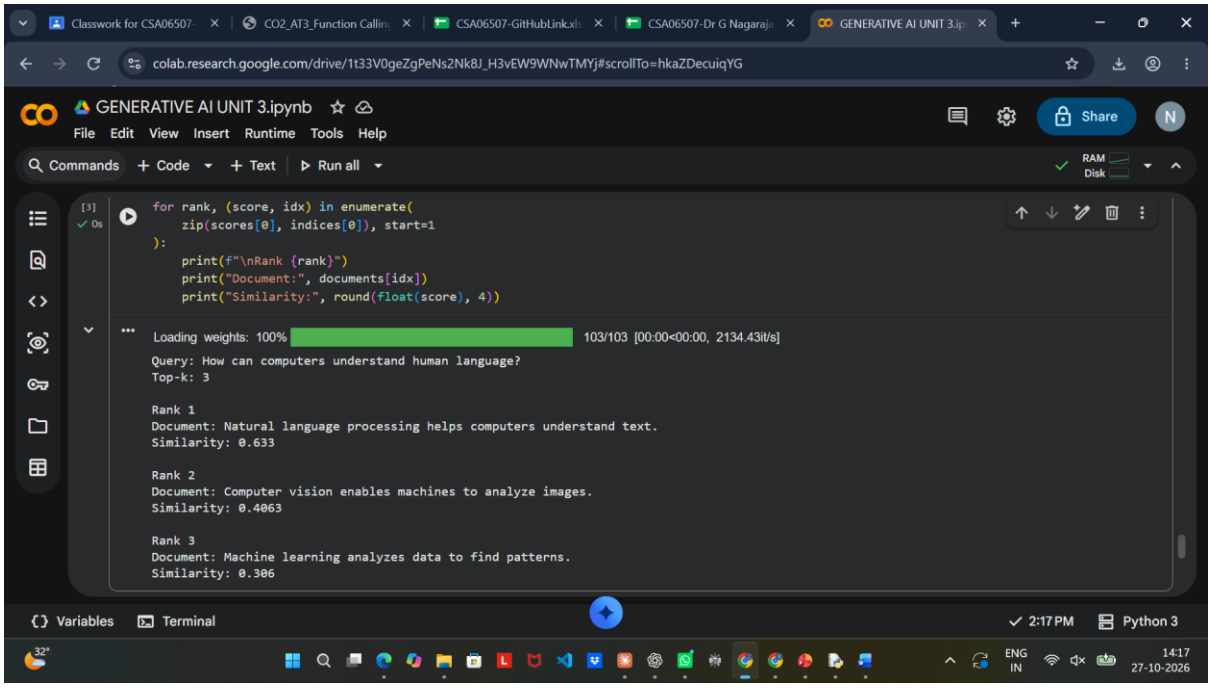
Document: [next most relevant document]

Similarity: [calculated score]

Rank 3

Document: [third most relevant document]

Similarity: [calculated score]



RESULT

Documents were stored in a FAISS vector database, and the top-k most relevant documents were retrieved successfully.